

AEGES-Q: Classical Cryptography Benchmark

Objective

This notebook evaluates classical cryptographic primitives suitable for integration into the AEGES-Q security framework.

The experiments establish a measurable classical cryptography baseline before introducing post-quantum cryptographic mechanisms.

The benchmark evaluates:

- Key generation latency
- Encryption latency
- Decryption latency
- Throughput
- Ciphertext overhead
- Repeated-operation performance

The results will be used to guide the production cryptographic layer of AEGES-Q and provide a baseline for later comparison with post-quantum cryptography.

```
# imports and configs
import os
import time
import statistics
from pathlib import Path

import numpy as np
import pandas as pd

from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.asymmetric.x25519 import (
    X25519PrivateKey,
)
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives import hashes

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

PROJECT_ROOT = Path.cwd().resolve().parents[1]

print("Project root:", PROJECT_ROOT)
print("Classical cryptography environment ready.")
```

Project root: C:\Projects\Aeges-Q
Classical cryptography environment ready.

```
import cryptography

print("Cryptography version:", cryptography.__version__)
print("AESGCM available:", AESGCM is not None)
print("X25519 available:", X25519PrivateKey is not None)
print("HKDF available:", HKDF is not None)
```

Cryptography version: 50.0.1
AESGCM available: True
X25519 available: True
HKDF available: True

1. AES-256-GCM Baseline

AES-256-GCM is used as the primary classical symmetric encryption primitive for the AEGES-Q baseline.

GCM provides authenticated encryption, combining:

- Confidentiality
- Integrity
- Authentication

A 256-bit AES key is generated for the experiment.

The implementation will be benchmarked for:

- Key generation
- Encryption
- Decryption
- Throughput
- Ciphertext overhead

```
# Generate a random 256-bit AES key
aes_key = AESGCM.generate_key(bit_length=256)

print("AES key length:", len(aes_key), "bytes")
print("AES key size:", len(aes_key) * 8, "bits")
```

AES key length: 32 bytes
AES key size: 256 bits

```
# Representative network/session payload
plaintext = (
    b"AEGES-Q network security session payload. "
    b"This data represents protected application traffic "
    b"processed by the classical cryptographic layer."
```

```

)

print("Plaintext size:", len(plaintext), "bytes")
print("Plaintext:", plaintext.decode())

Plaintext size: 140 bytes
Plaintext: AEGES-Q network security session payload. This data
represents protected application traffic processed by the classical
cryptographic layer.

# Initialize AES-GCM
aesgcm = AESGCM(aes_key)

# Generate a fresh 96-bit nonce
nonce = os.urandom(12)

# Optional authenticated metadata
associated_data = b"AEGES-Q|CLASSICAL|AES-256-GCM"

# Encrypt
ciphertext = aesgcm.encrypt(
    nonce,
    plaintext,
    associated_data
)

print("Plaintext size:", len(plaintext), "bytes")
print("Nonce size:", len(nonce), "bytes")
print("Ciphertext size:", len(ciphertext), "bytes")

# Decrypt
decrypted = aesgcm.decrypt(
    nonce,
    ciphertext,
    associated_data
)

print("\nDecryption successful:", decrypted == plaintext)
print("Decrypted data:", decrypted.decode())

Plaintext size: 140 bytes
Nonce size: 12 bytes
Ciphertext size: 156 bytes

Decryption successful: True
Decrypted data: AEGES-Q network security session payload. This data
represents protected application traffic processed by the classical
cryptographic layer.

```

1.1 Authentication and Tamper Detection

AES-256-GCM provides authenticated encryption.

To verify the integrity protection experimentally, the ciphertext will be modified before decryption.

A correctly implemented AES-GCM operation should reject the modified ciphertext rather than returning corrupted plaintext.

```
from cryptography.exceptions import InvalidTag

# Create a modified copy of the ciphertext
tampered_ciphertext = bytearray(ciphertext)
tampered_ciphertext[0] ^= 1
tampered_ciphertext = bytes(tampered_ciphertext)

# Attempt decryption of tampered ciphertext
try:
    aesgcm.decrypt(
        nonce,
        tampered_ciphertext,
        associated_data
    )
    print("Tampering detection: FAILED")
except InvalidTag:
    print("Tampering detection: PASSED")
    print("AES-GCM rejected the modified ciphertext.")

Tampering detection: PASSED
AES-GCM rejected the modified ciphertext.
```

2. AES-256-GCM Performance Benchmark

The benchmark measures AES-256-GCM performance across multiple payload sizes.

Metrics:

- Encryption latency
- Decryption latency
- Throughput
- Ciphertext overhead

Each operation uses a fresh nonce to maintain correct AES-GCM usage.

```
# Benchmark configuration

PAYLOAD_SIZES = [
    1024,          # 1 KB
```

```

    4096,          # 4 KB
    16384,        # 16 KB
    65536,        # 64 KB
    1048576       # 1 MB
]

WARMUP_RUNS = 20
BENCHMARK_RUNS = 500

print("Payload sizes:", PAYLOAD_SIZES)
print("Warm-up runs:", WARMUP_RUNS)
print("Benchmark runs:", BENCHMARK_RUNS)

Payload sizes: [1024, 4096, 16384, 65536, 1048576]
Warm-up runs: 20
Benchmark runs: 500

# AES-256-GCM performance benchmark

benchmark_results = []

for payload_size in PAYLOAD_SIZES:
    payload = os.urandom(payload_size)

    # Warm-up
    for _ in range(WARMUP_RUNS):
        nonce = os.urandom(12)
        encrypted = aesgcm.encrypt(nonce, payload, associated_data)
        aesgcm.decrypt(nonce, encrypted, associated_data)

    encryption_times = []
    decryption_times = []

    # Benchmark
    for _ in range(BENCHMARK_RUNS):
        nonce = os.urandom(12)

        # Encryption
        start = time.perf_counter()
        encrypted = aesgcm.encrypt(nonce, payload, associated_data)
        encryption_times.append(time.perf_counter() - start)

        # Decryption
        start = time.perf_counter()
        decrypted = aesgcm.decrypt(nonce, encrypted, associated_data)
        decryption_times.append(time.perf_counter() - start)

        # Safety check
        assert decrypted == payload

    encryption_median = statistics.median(encryption_times)

```

```

decryption_median = statistics.median(decryption_times)

encryption_mean = statistics.mean(encryption_times)
decryption_mean = statistics.mean(decryption_times)

encryption_throughput = payload_size / encryption_median / (1024
** 2)
decryption_throughput = payload_size / decryption_median / (1024
** 2)

benchmark_results.append({
    "payload_size_bytes": payload_size,
    "encryption_median_ms": encryption_median * 1000,
    "decryption_median_ms": decryption_median * 1000,
    "encryption_mean_ms": encryption_mean * 1000,
    "decryption_mean_ms": decryption_mean * 1000,
    "encryption_throughput_Mbps": encryption_throughput,
    "decryption_throughput_Mbps": decryption_throughput,
    "ciphertext_overhead_bytes": len(encrypted) - payload_size,
})

```

```
benchmark_df = pd.DataFrame(benchmark_results)
```

```
benchmark_df
```

	payload_size_bytes	encryption_median_ms	decryption_median_ms \
0	1024	0.00760	0.0076
1	4096	0.01030	0.0101
2	16384	0.01700	0.0168
3	65536	0.05235	0.0515
4	1048576	1.52695	1.4978

	encryption_mean_ms	decryption_mean_ms	encryption_throughput_Mbps \
0	0.008366	0.007638	128.495071
1	0.012956	0.011367	379.248971
2	0.018669	0.019613	919.120090
3	0.406333	0.175184	1193.887499
4	2.036420	1.663254	654.900298

	decryption_throughput_Mbps	ciphertext_overhead_bytes
0	128.495071	16
1	386.756710	16
2	930.058887	16
3	1213.592594	16
4	667.645882	16

```
# Format benchmark results for analysis
```

```
display_df = benchmark_df.copy()
```

```
display_df["payload_size_kb"] = (  
    display_df["payload_size_bytes"] / 1024  
)
```

```
display_df = display_df[  
    [  
        "payload_size_bytes",  
        "payload_size_kb",  
        "encryption_median_ms",  
        "decryption_median_ms",  
        "encryption_throughput_MBps",  
        "decryption_throughput_MBps",  
        "ciphertext_overhead_bytes",  
    ]  
]
```

```
display_df
```

	payload_size_bytes	payload_size_kb	encryption_median_ms	\
0	1024	1.0	0.00760	
1	4096	4.0	0.01030	
2	16384	16.0	0.01700	
3	65536	64.0	0.05235	
4	1048576	1024.0	1.52695	

	decryption_median_ms	encryption_throughput_MBps	\
0	0.0076	128.495071	
1	0.0101	379.248971	
2	0.0168	919.120090	
3	0.0515	1193.887499	
4	1.4978	654.900298	

	decryption_throughput_MBps	ciphertext_overhead_bytes
0	128.495071	16
1	386.756710	16
2	930.058887	16
3	1213.592594	16
4	667.645882	16

```
# Calculate ciphertext overhead percentage
```

```
benchmark_df["overhead_percentage"] = (  
    benchmark_df["ciphertext_overhead_bytes"]  
    / benchmark_df["payload_size_bytes"]  
    * 100  
)
```

```

overhead_df = benchmark_df[
    [
        "payload_size_bytes",
        "ciphertext_overhead_bytes",
        "overhead_percentage"
    ]
].copy()
overhead_df

```

	payload_size_bytes	ciphertext_overhead_bytes	overhead_percentage
0	1024	16	1.562500
1	4096	16	0.390625
2	16384	16	0.097656
3	65536	16	0.024414
4	1048576	16	0.001526

```

# Save AES-256-GCM benchmark results

crypto_artifacts_dir = PROJECT_ROOT / "artifacts" / "crypto"
crypto_artifacts_dir.mkdir(parents=True, exist_ok=True)

aes_benchmark_path = (
    crypto_artifacts_dir / "aes_256_gcm_benchmark.csv"
)

benchmark_df.to_csv(aes_benchmark_path, index=False)

print("Saved:", aes_benchmark_path)
print("Rows:", len(benchmark_df))

Saved: C:\Projects\Aeges-Q\artifacts\crypto\aes_256_gcm_benchmark.csv
Rows: 5

```

3. X25519 + HKDF Key Establishment

X25519 is used for ephemeral elliptic-curve Diffie-Hellman key agreement.

HKDF-SHA256 is then used to derive a fixed-length application/session key from the shared secret.

The experiment verifies:

- X25519 key generation
- Shared-secret agreement
- HKDF session-key derivation
- Agreement between both parties
- Key-establishment latency

```
# X25519 + HKDF configuration
```

```
HKDF_INFO = b"AEGES-Q|CLASSICAL|SESSION-KEY"
```

```
DERIVED_KEY_LENGTH = 32
```

```
print("Key agreement: X25519")
```

```
print("KDF: HKDF-SHA256")
```

```
print("Derived key length:", DERIVED_KEY_LENGTH, "bytes")
```

```
Key agreement: X25519
```

```
KDF: HKDF-SHA256
```

```
Derived key length: 32 bytes
```

```
# Generate ephemeral X25519 key pairs for two parties
```

```
client_private_key = X25519PrivateKey.generate()
```

```
client_public_key = client_private_key.public_key()
```

```
server_private_key = X25519PrivateKey.generate()
```

```
server_public_key = server_private_key.public_key()
```

```
print("Client key pair generated.")
```

```
print("Server key pair generated.")
```

```
Client key pair generated.
```

```
Server key pair generated.
```

```
# Perform X25519 Diffie-Hellman key agreement
```

```
client_shared_secret = client_private_key.exchange(server_public_key)
```

```
server_shared_secret = server_private_key.exchange(client_public_key)
```

```
print("Client shared secret length:", len(client_shared_secret),  
      "bytes")
```

```
print("Server shared secret length:", len(server_shared_secret),  
      "bytes")
```

```
print(  
    "Shared secrets match:",  
    client_shared_secret == server_shared_secret  
)
```

```
Client shared secret length: 32 bytes
```

```
Server shared secret length: 32 bytes
```

```
Shared secrets match: True
```

```
# Derive a 256-bit session key from the shared secret using HKDF-SHA256
```

```
client_session_key = HKDF(  
    algorithm=hashes.SHA256(),  
    length=DERIVED_KEY_LENGTH,
```

```

        salt=None,
        info=HKDF_INFO,
    ).derive(client_shared_secret)

server_session_key = HKDF(
    algorithm=hashes.SHA256(),
    length=DERIVED_KEY_LENGTH,
    salt=None,
    info=HKDF_INFO,
).derive(server_shared_secret)

print("Client session key length:", len(client_session_key), "bytes")
print("Server session key length:", len(server_session_key), "bytes")
print(
    "Derived session keys match:",
    client_session_key == server_session_key
)

```

```

Client session key length: 32 bytes
Server session key length: 32 bytes
Derived session keys match: True

```

X25519 + HKDF key-establishment benchmark

```

KEY_EXCHANGE_WARMUP_RUNS = 20
KEY_EXCHANGE_BENCHMARK_RUNS = 500

```

```
key_exchange_times = []
```

Warm-up

```

for _ in range(KEY_EXCHANGE_WARMUP_RUNS):
    client_private = X25519PrivateKey.generate()
    client_public = client_private.public_key()

    server_private = X25519PrivateKey.generate()
    server_public = server_private.public_key()

    client_secret = client_private.exchange(server_public)
    server_secret = server_private.exchange(client_public)

    assert client_secret == server_secret

    client_key = HKDF(
        algorithm=hashes.SHA256(),
        length=DERIVED_KEY_LENGTH,
        salt=None,
        info=HKDF_INFO,
    ).derive(client_secret)

    server_key = HKDF(
        algorithm=hashes.SHA256(),

```

```

        length=DERIVED_KEY_LENGTH,
        salt=None,
        info=HKDF_INFO,
    ).derive(server_secret)

    assert client_key == server_key

# Benchmark
for _ in range(KEY_EXCHANGE_BENCHMARK_RUNS):
    start = time.perf_counter()

    # Generate ephemeral key pairs
    client_private = X25519PrivateKey.generate()
    client_public = client_private.public_key()

    server_private = X25519PrivateKey.generate()
    server_public = server_private.public_key()

    # X25519 key agreement
    client_secret = client_private.exchange(server_public)
    server_secret = server_private.exchange(client_public)

    # HKDF-SHA256 key derivation
    client_key = HKDF(
        algorithm=hashes.SHA256(),
        length=DERIVED_KEY_LENGTH,
        salt=None,
        info=HKDF_INFO,
    ).derive(client_secret)

    server_key = HKDF(
        algorithm=hashes.SHA256(),
        length=DERIVED_KEY_LENGTH,
        salt=None,
        info=HKDF_INFO,
    ).derive(server_secret)

    elapsed = time.perf_counter() - start

    assert client_secret == server_secret
    assert client_key == server_key

    key_exchange_times.append(elapsed)

key_exchange_median_ms = statistics.median(key_exchange_times) * 1000
key_exchange_mean_ms = statistics.mean(key_exchange_times) * 1000
key_exchange_std_ms = statistics.stdev(key_exchange_times) * 1000

print("X25519 + HKDF benchmark")

```

```

print("-----")
print(f"Runs: {KEY_EXCHANGE_BENCHMARK_RUNS}")
print(f"Median latency: {key_exchange_median_ms:.4f} ms")
print(f"Mean latency: {key_exchange_mean_ms:.4f} ms")
print(f"Std deviation: {key_exchange_std_ms:.4f} ms")
print("Session-key agreement verified: True")

```

X25519 + HKDF benchmark

```

Runs: 500
Median latency: 0.6148 ms
Mean latency: 0.6650 ms
Std deviation: 0.3608 ms
Session-key agreement verified: True

```

Save X25519 + HKDF benchmark results

```

key_exchange_benchmark = pd.DataFrame([
    "algorithm": "X25519 + HKDF-SHA256",
    "runs": KEY_EXCHANGE_BENCHMARK_RUNS,
    "median_latency_ms": key_exchange_median_ms,
    "mean_latency_ms": key_exchange_mean_ms,
    "std_latency_ms": key_exchange_std_ms,
    "derived_key_bytes": DERIVED_KEY_LENGTH,
    "session_key_agreement_verified": True,
])

```

```

key_exchange_path = (
    crypto_artifacts_dir / "x25519_hkdf_benchmark.csv"
)

```

```

key_exchange_benchmark.to_csv(
    key_exchange_path,
    index=False
)

```

```

print("Saved:", key_exchange_path)
print()
display(key_exchange_benchmark)

```

Saved: C:\Projects\Aeges-Q\artifacts\crypto\x25519_hkdf_benchmark.csv

	algorithm	runs	median_latency_ms	mean_latency_ms	\
0	X25519 + HKDF-SHA256	500	0.6148	0.665038	
	std_latency_ms	derived_key_bytes	session_key_agreement_verified		
0	0.360806	32	True		

4. Classical Cryptography Baseline

The experiments establish the classical cryptographic baseline for AEGES-Q.

AES-256-GCM provides authenticated symmetric encryption, while X25519 combined with HKDF-SHA256 provides ephemeral key agreement and session-key derivation.

These measurements will serve as the reference point for later post-quantum cryptography experiments.

```
# Combine the classical cryptography benchmark results
```

```
classical_crypto_summary = pd.DataFrame([
    {
        "component": "AES-256-GCM",
        "operation": "Encryption",
        "median_latency_ms":
benchmark_df["encryption_median_ms"].mean(),
        "mean_latency_ms": benchmark_df["encryption_mean_ms"].mean(),
        "key_size_bytes": 32,
        "fixed_overhead_bytes": 16,
    },
    {
        "component": "AES-256-GCM",
        "operation": "Decryption",
        "median_latency_ms":
benchmark_df["decryption_median_ms"].mean(),
        "mean_latency_ms": benchmark_df["decryption_mean_ms"].mean(),
        "key_size_bytes": 32,
        "fixed_overhead_bytes": 16,
    },
    {
        "component": "X25519 + HKDF-SHA256",
        "operation": "Key establishment",
        "median_latency_ms": key_exchange_median_ms,
        "mean_latency_ms": key_exchange_mean_ms,
        "key_size_bytes": DERIVED_KEY_LENGTH,
        "fixed_overhead_bytes": None,
    },
])
```

```
classical_crypto_summary
```

	component	operation	median_latency_ms	\
0	AES-256-GCM	Encryption	0.32284	
1	AES-256-GCM	Decryption	0.31676	
2	X25519 + HKDF-SHA256	Key establishment	0.61480	
	mean_latency_ms	key_size_bytes	fixed_overhead_bytes	
0	0.496549	32	16.0	

```

1          0.375411          32          16.0
2          0.665038          32          NaN

# Save combined classical cryptography summary

classical_summary_path = (
    crypto_artifacts_dir / "classical_crypto_baseline_summary.csv"
)

classical_crypto_summary.to_csv(
    classical_summary_path,
    index=False
)

print("Saved:", classical_summary_path)
print()
display(classical_crypto_summary)

Saved: C:\Projects\Aeges-Q\artifacts\crypto\
classical_crypto_baseline_summary.csv

```

	component	operation	median_latency_ms \
0	AES-256-GCM	Encryption	0.32284
1	AES-256-GCM	Decryption	0.31676
2	X25519 + HKDF-SHA256	Key establishment	0.61480

	mean_latency_ms	key_size_bytes	fixed_overhead_bytes
0	0.496549	32	16.0
1	0.375411	32	16.0
2	0.665038	32	NaN

5. AES-256 Key Generation Benchmark

AES-256 key generation is benchmarked independently to characterize the cost of generating fresh 256-bit symmetric keys.

The benchmark uses repeated random key generation and reports median, mean, and standard deviation of generation latency.

```

# AES-256 key-generation benchmark

AES_KEYGEN_WARMUP_RUNS = 20
AES_KEYGEN_BENCHMARK_RUNS = 500

aes_keygen_times = []

# Warm-up
for _ in range(AES_KEYGEN_WARMUP_RUNS):
    AESGCM.generate_key(bit_length=256)

```

```

# Benchmark
for _ in range(AES_KEYGEN_BENCHMARK_RUNS):
    start = time.perf_counter()

    key = AESGCM.generate_key(bit_length=256)

    elapsed = time.perf_counter() - start
    aes_keygen_times.append(elapsed)

    assert len(key) == 32

aes_keygen_median_ms = statistics.median(aes_keygen_times) * 1000
aes_keygen_mean_ms = statistics.mean(aes_keygen_times) * 1000
aes_keygen_std_ms = statistics.stdev(aes_keygen_times) * 1000

print("AES-256 key-generation benchmark")
print("-----")
print(f"Runs: {AES_KEYGEN_BENCHMARK_RUNS}")
print(f"Median latency: {aes_keygen_median_ms:.6f} ms")
print(f"Mean latency: {aes_keygen_mean_ms:.6f} ms")
print(f"Std deviation: {aes_keygen_std_ms:.6f} ms")
print("Generated key size: 32 bytes")

AES-256 key-generation benchmark
-----
Runs: 500
Median latency: 0.003500 ms
Mean latency: 0.003527 ms
Std deviation: 0.000778 ms
Generated key size: 32 bytes

```

6. X25519 Key Generation and Agreement Benchmarks

The X25519 component is benchmarked at two levels:

- Ephemeral key-pair generation
- Shared-secret computation

This separates the cost of establishing the classical key agreement from the subsequent HKDF session-key derivation.

```

# X25519 key-generation and shared-secret benchmarks

X25519_WARMUP_RUNS = 20
X25519_BENCHMARK_RUNS = 500

x25519_keygen_times = []
x25519_exchange_times = []

```

```

# Warm-up
for _ in range(X25519_WARMUP_RUNS):
    client_private = X25519PrivateKey.generate()
    client_public = client_private.public_key()

    server_private = X25519PrivateKey.generate()
    server_public = server_private.public_key()

    client_private.exchange(server_public)
    server_private.exchange(client_public)

# Benchmark
for _ in range(X25519_BENCHMARK_RUNS):

    # Key-pair generation
    start = time.perf_counter()

    client_private = X25519PrivateKey.generate()
    client_public = client_private.public_key()

    server_private = X25519PrivateKey.generate()
    server_public = server_private.public_key()

    keygen_elapsed = time.perf_counter() - start
    x25519_keygen_times.append(keygen_elapsed)

    # Shared-secret computation
    start = time.perf_counter()

    client_secret = client_private.exchange(server_public)
    server_secret = server_private.exchange(client_public)

    exchange_elapsed = time.perf_counter() - start
    x25519_exchange_times.append(exchange_elapsed)

    assert client_secret == server_secret
    assert len(client_secret) == 32

x25519_keygen_median_ms = statistics.median(x25519_keygen_times) *
1000
x25519_keygen_mean_ms = statistics.mean(x25519_keygen_times) * 1000
x25519_keygen_std_ms = statistics.stdev(x25519_keygen_times) * 1000

x25519_exchange_median_ms = statistics.median(x25519_exchange_times) *
1000
x25519_exchange_mean_ms = statistics.mean(x25519_exchange_times) *
1000
x25519_exchange_std_ms = statistics.stdev(x25519_exchange_times) *
1000

```

```

print("X25519 key-generation benchmark")
print("-----")
print(f"Runs: {X25519_BENCHMARK_RUNS}")
print(f"Median latency: {x25519_keygen_median_ms:.6f} ms")
print(f"Mean latency:    {x25519_keygen_mean_ms:.6f} ms")
print(f"Std deviation:    {x25519_keygen_std_ms:.6f} ms")

print("\nX25519 shared-secret benchmark")
print("-----")
print(f"Runs: {X25519_BENCHMARK_RUNS}")
print(f"Median latency: {x25519_exchange_median_ms:.6f} ms")
print(f"Mean latency:    {x25519_exchange_mean_ms:.6f} ms")
print(f"Std deviation:    {x25519_exchange_std_ms:.6f} ms")
print("Shared-secret agreement verified: True")

```

X25519 key-generation benchmark

```

-----
Runs: 500
Median latency: 0.193350 ms
Mean latency:   0.237919 ms
Std deviation:  0.103676 ms

```

X25519 shared-secret benchmark

```

-----
Runs: 500
Median latency: 0.160000 ms
Mean latency:   0.187037 ms
Std deviation:  0.077712 ms
Shared-secret agreement verified: True

```

HKDF-SHA256 key-derivation benchmark

```

HKDF_WARMUP_RUNS = 20
HKDF_BENCHMARK_RUNS = 500

```

```

hkdf_times = []

```

```

# Use a representative 32-byte X25519 shared secret
hkdf_input = os.urandom(32)

```

Warm-up

```

for _ in range(HKDF_WARMUP_RUNS):
    derived_key = HKDF(
        algorithm=hashes.SHA256(),
        length=DERIVED_KEY_LENGTH,
        salt=None,
        info=HKDF_INFO,
    ).derive(hkdf_input)

```

```

    assert len(derived_key) == DERIVED_KEY_LENGTH

# Benchmark
for _ in range(HKDF_BENCHMARK_RUNS):
    start = time.perf_counter()

    derived_key = HKDF(
        algorithm=hashes.SHA256(),
        length=DERIVED_KEY_LENGTH,
        salt=None,
        info=HKDF_INFO,
    ).derive(hkdf_input)

    elapsed = time.perf_counter() - start
    hkdf_times.append(elapsed)

    assert len(derived_key) == DERIVED_KEY_LENGTH

hkdf_median_ms = statistics.median(hkdf_times) * 1000
hkdf_mean_ms = statistics.mean(hkdf_times) * 1000
hkdf_std_ms = statistics.stdev(hkdf_times) * 1000

print("HKDF-SHA256 benchmark")
print("-----")
print(f"Runs: {HKDF_BENCHMARK_RUNS}")
print(f"Median latency: {hkdf_median_ms:.6f} ms")
print(f"Mean latency: {hkdf_mean_ms:.6f} ms")
print(f"Std deviation: {hkdf_std_ms:.6f} ms")
print("Derived key size: 32 bytes")

HKDF-SHA256 benchmark
-----
Runs: 500
Median latency: 0.016600 ms
Mean latency: 0.016004 ms
Std deviation: 0.004677 ms
Derived key size: 32 bytes

# Complete classical cryptography benchmark summary
classical_crypto_summary = pd.DataFrame([
    {
        "component": "AES-256-GCM",
        "operation": "Key generation",
        "median_latency_ms": aes_keygen_median_ms,
        "mean_latency_ms": aes_keygen_mean_ms,
        "std_latency_ms": aes_keygen_std_ms,
        "key_size_bytes": 32,
        "fixed_overhead_bytes": 0,
    }
])

```

```

    },
    {
        "component": "AES-256-GCM",
        "operation": "Encryption",
        "median_latency_ms":
benchmark_df["encryption_median_ms"].mean(),
        "mean_latency_ms": benchmark_df["encryption_mean_ms"].mean(),
        "std_latency_ms": None,
        "key_size_bytes": 32,
        "fixed_overhead_bytes": 16,
    },
    {
        "component": "AES-256-GCM",
        "operation": "Decryption",
        "median_latency_ms":
benchmark_df["decryption_median_ms"].mean(),
        "mean_latency_ms": benchmark_df["decryption_mean_ms"].mean(),
        "std_latency_ms": None,
        "key_size_bytes": 32,
        "fixed_overhead_bytes": 16,
    },
    {
        "component": "X25519",
        "operation": "Key generation",
        "median_latency_ms": x25519_keygen_median_ms,
        "mean_latency_ms": x25519_keygen_mean_ms,
        "std_latency_ms": x25519_keygen_std_ms,
        "key_size_bytes": 32,
        "fixed_overhead_bytes": None,
    },
    {
        "component": "X25519",
        "operation": "Shared-secret computation",
        "median_latency_ms": x25519_exchange_median_ms,
        "mean_latency_ms": x25519_exchange_mean_ms,
        "std_latency_ms": x25519_exchange_std_ms,
        "key_size_bytes": 32,
        "fixed_overhead_bytes": None,
    },
    {
        "component": "HKDF-SHA256",
        "operation": "Key derivation",
        "median_latency_ms": hkdf_median_ms,
        "mean_latency_ms": hkdf_mean_ms,
        "std_latency_ms": hkdf_std_ms,
        "key_size_bytes": 32,
        "fixed_overhead_bytes": None,
    },
    {

```

```

        "component": "X25519 + HKDF-SHA256",
        "operation": "Complete key establishment",
        "median_latency_ms": key_exchange_median_ms,
        "mean_latency_ms": key_exchange_mean_ms,
        "std_latency_ms": key_exchange_std_ms,
        "key_size_bytes": 32,
        "fixed_overhead_bytes": None,
    },
]
display(classical_crypto_summary)

```

	component	operation	median_latency_ms
0	AES-256-GCM	Key generation	0.00350
1	AES-256-GCM	Encryption	0.32284
2	AES-256-GCM	Decryption	0.31676
3	X25519	Key generation	0.19335
4	X25519	Shared-secret computation	0.16000
5	HKDF-SHA256	Key derivation	0.01660
6	X25519 + HKDF-SHA256	Complete key establishment	0.61480

	mean_latency_ms	std_latency_ms	key_size_bytes
0	0.003527	0.000778	32
1	0.496549	NaN	32
2	0.375411	NaN	32
3	0.237919	0.103676	32
4	0.187037	0.077712	32
5	0.016004	0.004677	32
6	0.665038	0.360806	32

```

# Save final classical cryptography benchmark summary
classical_summary_path = (
    crypto_artifacts_dir / "classical_crypto_baseline_summary.csv"
)

```

```
classical_crypto_summary.to_csv(  
    classical_summary_path,  
    index=False  
)  
  
print("Saved:", classical_summary_path)  
  
Saved: C:\Projects\Aeges-Q\artifacts\crypto\  
classical_crypto_baseline_summary.csv
```

7. Classical Cryptography Baseline Validation

The classical cryptography baseline has been evaluated across the cryptographic operations required by the initial AEGES-Q security layer.

AES-256-GCM was evaluated for authenticated encryption performance, including encryption, decryption, throughput, ciphertext overhead, and tamper detection.

X25519 was evaluated for ephemeral key generation and shared-secret computation.

HKDF-SHA256 was evaluated for session-key derivation.

The complete X25519 + HKDF-SHA256 key-establishment flow was also benchmarked and verified to produce matching session keys between both parties.

These measurements establish the classical reference baseline for later comparison with post-quantum cryptographic mechanisms.

```
# Final validation checks  
  
assert benchmark_df["ciphertext_overhead_bytes"].eq(16).all()  
assert benchmark_df["payload_size_bytes"].gt(0).all()  
  
assert aes_keygen_median_ms > 0  
assert x25519_keygen_median_ms > 0  
assert x25519_exchange_median_ms > 0  
assert hkdf_median_ms > 0  
assert key_exchange_median_ms > 0  
  
assert DERIVED_KEY_LENGTH == 32  
  
print("Classical cryptography benchmark validation: PASSED")  
print()  
print("AES-256-GCM: validated")  
print("X25519: validated")  
print("HKDF-SHA256: validated")  
print("X25519 + HKDF-SHA256: validated")  
print("Benchmark artifacts: generated")
```

Classical cryptography benchmark validation: PASSED

AES-256-GCM: validated

X25519: validated

HKDF-SHA256: validated

X25519 + HKDF-SHA256: validated

Benchmark artifacts: generated

8. Conclusion

The classical cryptography experiments establish the baseline cryptographic configuration selected for AEGES-Q.

AES-256-GCM provides authenticated symmetric encryption with a fixed 16-byte authentication tag and measurable throughput across payload sizes from 1 KB to 1 MB.

X25519 provides ephemeral classical key agreement, while HKDF-SHA256 derives the 256-bit session keys used by the application layer.

The complete X25519 + HKDF-SHA256 workflow was verified by independently performing the exchange from both parties and confirming identical derived session keys.

The resulting measurements form the classical cryptographic reference against which future post-quantum key-establishment mechanisms can be evaluated.

The benchmark is experimental and hardware-dependent; the reported latencies and throughput characterize the local execution environment rather than representing universal cryptographic performance.